

AI UTILIZATION

AI 활용 기술 문서

투기장에서는 칼보다 배팅이다

중세 콜로세움에서 팀의 승패를 예측하고 배팅하는 모바일 캐주얼 게임

기획 지식

프로그래밍

UI 아이콘

BGM



그림 1. 중세 콜로세움과 캐주얼 전투 콘셉트를 보여 주는 게임 대표 화면

활용 원칙

AI는 조사·초안·반복 작업을 보조했다. 게임 규칙과 구현 방향은 팀이 결정했고, 코드·UI·음악 결과는 담당자가 직접 검증한 뒤 반영하였다.

OVERVIEW

AI 활용 전체 구조

AI가 처리할 반복 작업과 개발자가 책임질 판단·검증을 분리

프로젝트의 AI 활용은 기획 지식 구조화, 개발 보조, 이미지·음악 제작, 개발자 검증의 네 단계로 구성하였다. 목적은 전체 제작을 자동화하는 것이 아니라 파일 탐색, 흐름 정리, 반복 수정, 테스트 조건 작성에 드는 시간을 줄이는 것이었다.



최종 책임

AI가 작성한 결과를 그대로 최종 결과로 인정하지 않았다. 위키는 코드와 테스트를 근거로 갱신했고, 구현은 자동 테스트와 Unity Play Mode로 확인했다. 아이콘은 실제 UI 크기에서 가독성을 확인하고, 음악은 장면과 길이에 맞는지 사람이 비교하도록 하였다.

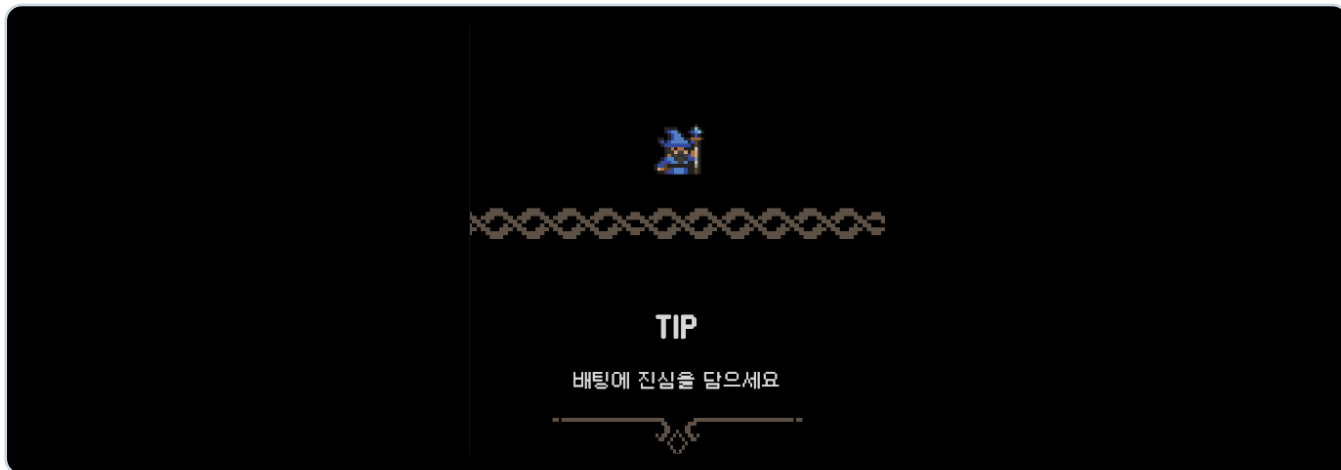


그림 4. AI 보조 작업을 통합한 뒤 Unity에서 직접 확인한 인게임 로딩 화면

PLANNING

LLM 기반 구현 위키

문서와 실제 Unity 구현이 함께 갱신되는 게임 기획 아카이브

Unity 기능은 Script 하나로 끝나지 않는다. ScriptableObject 데이터, Scene-Prefab 연결, Inspector 설정, 테스트가 함께 동작한다. 이를 반영해 게임 개요, 라운드와 베팅, 자동 전투, 전투 아이템, 콘텐츠 데이터, 저장·보상, UI 흐름, 런타임 구조를 Markdown-Obsidian 위키로 관리하였다.

근거 확인 순서

1 현재 코드·데이터

2 Scene-Prefab 연결

3 테스트

4 최근 Git 기록

5 기존 문서·해석

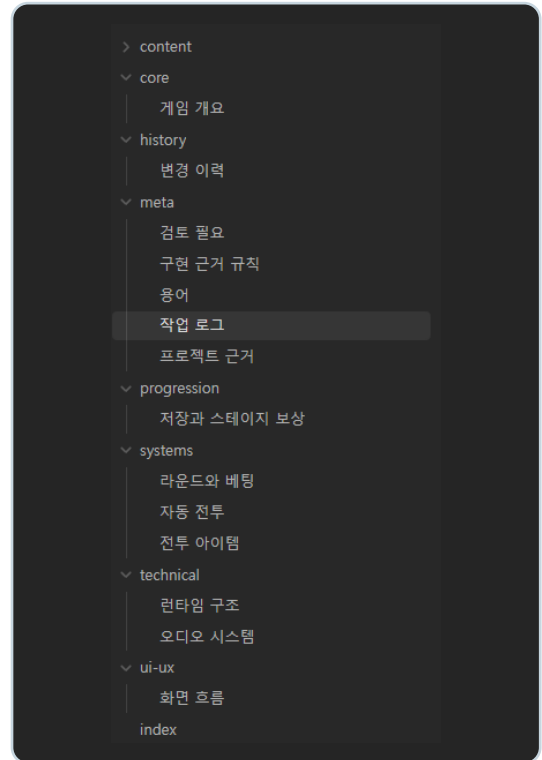
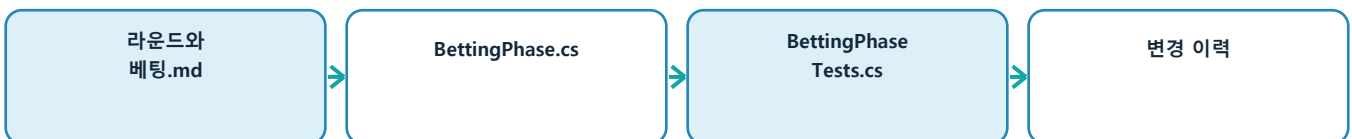


그림 2. Obsidian 구현 위키 목차

문서와 구현 근거 연결



상태 관리 원칙

실행 경로가 확인된 내용만 implemented로 기록하였다. 선언은 있지만 연결을 확인하지 못한 내용은 partial 또는 review로 분리하였다. 구현과 기존 문서가 충돌하면 현재 구현을 우선하되, 의도와 다르게 보이는 내용은 임의로 확정하지 않았다.

PROMPTS

위키용 주요 프롬프트와 갱신 과정

문장 생성보다 사실을 판단하는 기준을 프롬프트로 고정

2026-08-10 — BGM Catalog-씬-페이즈 연결

- 추가된 5개 BGM를 SoundCatalog의 의미 기반 ID로 연결하고 Master/BGM/SFX AudioManager를 구성했다.
- Title 씬의 영속 Managers에 SoundManager를 등록하고 Title/Lobby/MainGame 씬 매핑을 적용했다.
- RoundManager가 Betting Phase에서 베팅곡, Combat Phase에서 전투곡 세 개 중 하나를 재생 하도록 갱신했다. Result Phase에서는 전투곡을 유지한다.
- 오디오 시스템, 변경 이력, 현재 구현 색인을 최종 구현 근거에 맞춰 갱신했다. 확인 근거: Assets/Scripts/Managers/SoundManager.cs, Assets/Scripts/Managers/RoundManager.cs, Assets/ScriptableObject/SoundCatalog.asset, Assets/Audio/InTheArenaAudioMixer.mixer, Assets/Scenes/Title.unity.

2026-08-10 — 보상 아이콘 비행 연출 로딩 전환 안정화

- UI_FlyingRewardEffect의 보상 아이콘 대기를 0.25초로 단축했다.
- UI_LobbyHeader가 LoadingProgressService의 로딩 완료 후 일회성 스테이지 클리어 보상 연출을 소비하도록 수정했다. AsyncSceneLoader의 전환 종료 트윈 처리와 겹쳐 아이콘 인내가 잔류하던 문제를 방지한다.
- 피징과 스테이지 보상 화면 종료, 변경 이력의 구현 근거와 UI 흐름을 갱신했다. 확인 근거: Assets/Scripts/UI/UI_FlyingRewardEffect.cs, Assets/Scripts/UI/UI_LobbyHeader.cs, Assets/Scripts/Util/AsyncSceneLoader.cs, Assets/Scripts/Util/LoadingProgressService.cs.

운영 지시

현재 Unity 프로젝트의 실제 구현을 기준으로 한국어 Markdown 위키를 관리한다. 코드-데이터, Scene-Prefab, 테스트, Git 기록 순으로 확인한다. 근거가 부족하면 partial 또는 review로 표시한다. 같은 설명은 복사하지 않고 Obsidian 링크로 연결한다.

기능 조사 프롬프트

현재 구현을 먼저 조사하고 바로 문서를 작성하지 마라. 관련 Script, ScriptableObject, Scene-Prefab 연결, 테스트를 찾아 실행 경로를 정리해라. 확인된 사실과 해석을 구분하고, 최종 문서에는 근거 경로와 확인일을 기록해라.

구현 후 갱신 프롬프트

최종 git diff와 변경 파일을 기준으로 위키 영향 여부를 확인해라. 게임 규칙이나 데이터 의미가 바뀌었다면 시스템 문서와 변경 이력을 갱신한다. 외부 동작이 바뀌지 않은 리팩터링은 사양 변경으로 기록하지 않는다. 갱신 뒤 내부 링크와 중복 설명을 검사한다.

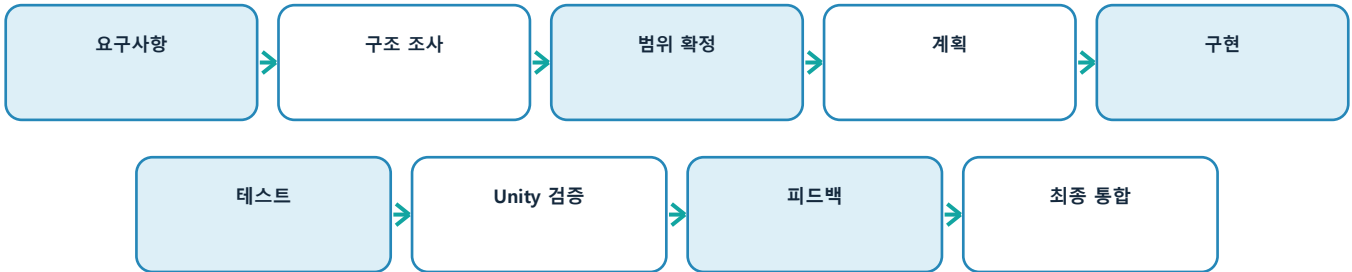
실제 갱신 기록

그림 3은 BGM Catalog 연결과 로딩 전환 안정화 작업을 위키 작업 로그에 기록한 화면이다. 변경 내용뿐 아니라 확인한 Script-Scene-Asset 경로까지 함께 남겨, 이후 문서가 어떤 구현을 근거로 작성됐는지 다시 확인할 수 있게 하였다.

PROGRAMMING

일관된 AI 작업 과정과 병렬 개발

조사부터 검증까지의 순서를 고정해 작업 과정을 확인 가능하게 구성



사용자가 원하는 결과와 실제 관찰한 현상을 먼저 설명하고, AI가 Script-Prefab-ScriptableObject-테스트를 조사한 뒤 수정 대상과 확인 대상만 구분하게 했다. 화면 배치와 입력처럼 코드만으로 판단하기 어려운 부분은 팀원이 Unity에서 직접 실행하였다.

Git worktree를 이용한 병렬 작업



역할에 맞춘 AI 배치



효과

작업 폴더를 분리해 Compile-Play Mode-Scene 저장 충돌을 줄였고, 중요한 모델은 반복 검색보다 구조 판단과 코드 작성에 집중하도록 하였다.

DEBUGGING

원인 역추적과 수정 범위 조사

버그가 보이는 결과에서 시작해 Target·거리·이동·공격 판정을 반대로 추적



그림 5. 탐색·이동·공격 판정을 점검한 실제 전투 장면

Unit AI 사례

두 근접 Unit이 서로를 Target으로 잡고 가까이 이동하지만 일정 거리에서 공격하지 않는 문제가 있었다. AI에게 바로 수정을 요청하지 않고 적 탐색부터 공격 판정까지 전체 흐름을 확인하게 했다. 이동과 공격이 서로 다른 위치 기준을 사용했고, 근접 유닛은 Attack Range뿐 아니라 실제 접촉 거리와 이동 허용 오차도 고려해야 함을 확인했다.

적 탐색

Target 지정

거리 계산

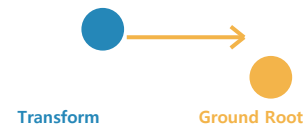
이동 위치

공격 판정

대표 지시

두 근접 Unit이 일정 거리에서 공격하지 않는다. 바로 수정하지 말고 Target 탐색부터 이동·공격 거리 계산까지 확인해라. Search Range, Attack Range, 실제 Unit 크기와 기준 Transform이 다르게 계산되는지도 확인하고, 근접·원거리 상황을 회귀 테스트로 남겨라.

기준점 차이



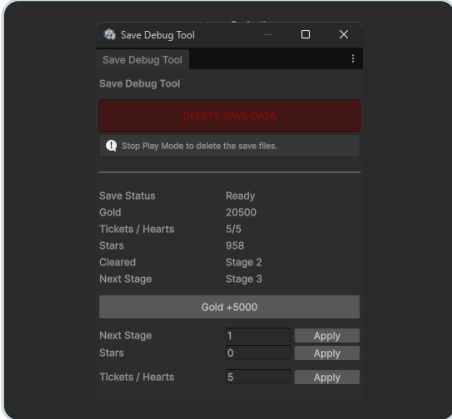
HP Bar 수정 범위

Inspector 값만 바꾸는 문제가 아니었다. Unit의 HitPosition을 화면 좌표로 바꾼 뒤 Runtime에서 위치와 크기를 다시 적용하고 있어 Script 계산과 Prefab 설정을 함께 확인했다. AI가 줄여 준 시간은 코드 입력보다 값이 다시 바뀌는 위치를 찾는 시간이었다.

TOOLS & SAFETY

개발 도구 제작과 안정성 보완

반복 테스트 환경을 자동화하고 정상 경로 밖의 입력도 회귀 테스트로 고정



UnitSystemMigration

구버전 Unit Prefab을 새 구조로 일괄 변환하고 누락된 Component와 기준점을 정리하였다.

RoundDataEditor

Team A/B의 2×3 배치를 실제 전장과 비슷한 Grid 형태로 편집하도록 Inspector를 구성하였다.

테스트 상태 생성

원하는 재화·스테이지 상태를 즉시 만들어 반복 플레이 시간을 줄였다.

구버전 일괄 변환

Unit Prefab에 같은 구조와 Component 규칙을 적용해 누락을 줄였다.

데이터 편집 개선

배열 대신 실제 배치에 가까운 Grid로 RoundData를 수정하였다.

Battle Item Drag 예외 처리

Meteor와 Mercenary는 전장 위치를 지정해야 하므로 정상 입력만 확인해서는 부족했다. 처음 누른 Pointer ID를 저장해 다른 입력을 무시하고, 전장 밖에서는 DraggingInvalid 상태로 전환하였다. Targeting 중 UI가 사라지면 상태와 전투 속도를 복구하고, Mercenary 세 명이 모두 전장 안에 생성되는지도 계산하였다.



검증 항목

전장 안·밖 Drag
다른 Pointer의 입력 무시
Targeting 취소 시 상태 복구
Mercenary 생성 범위 계산
실패 시 Gold·사용 상태 복구

검증한 예외

다른 Pointer의 PointerUp · 전장 밖 Drag · Targeting 중 비활성화 · 아이템 실패 시 Gold와 사용 상태 복구

IMAGE GENERATION

ChatGPT를 활용한 32×32 UI 아이콘 제작

기존 UI 시트를 기준으로 스타일을 고정하고 수정 요청을 단계적으로 반복



그림 8. 아이콘이 적용된 베팅 화면

대표 프롬프트

32×32 픽셀 아이콘. 중세 콜로세움에서 팀에 베팅해 돈을 버는 모바일 캐주얼 게임이며, 3 Match Puzzle처럼 밝고 읽기 쉬운 스타일을 사용한다. 작은 크기에서 구분되는 실루엣, 투명 배경 PNG, 텍스트·워터마크 제외 조건을 유지한다.



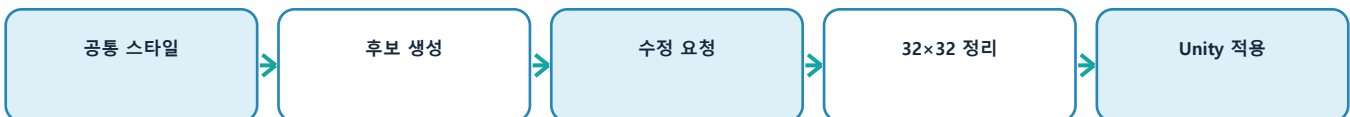
그림 9. 재화-음션 아이콘의 실제 상단 UI 적용

수정과 선별

초안에서 문양과 색상이 UI 목적에 맞지 않으면 입장권의 월계관, 별 모양, 지폐 색상처럼 수정 조건을 구체적으로 다시 전달하였다. 결과는 사람이 작은 화면에서 비교해 선택하였다.

실제 적용 과정

ChatGPT가 생성한 후보를 그대로 사용하지 않고 투명 배경과 가장자리를 확인한 뒤 32×32 크기로 정리하였다. Unity에서는 픽셀 경계가 흐려지지 않도록 Point Filter를 적용하고, 베팅 화면과 상단 HUD에서 실루엣과 색상 구분이 유지되는지 확인하였다.



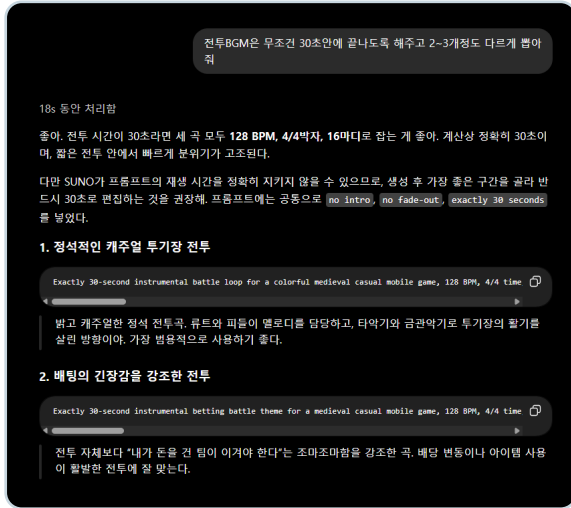
사람이 담당한 최종 판단

아이콘의 의미, 수정 방향, 실제 UI 배치와 최종 채택 여부는 개발자가 결정하였다. AI는 동일한 시각 조건에서 여러 후보를 빠르게 비교하기 위한 제작 보조 도구로 사용하였다.

MUSIC PROMPTING

ChatGPT와 SUNO를 활용한 BGM 제작

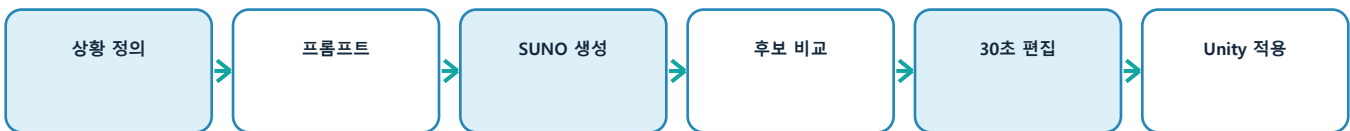
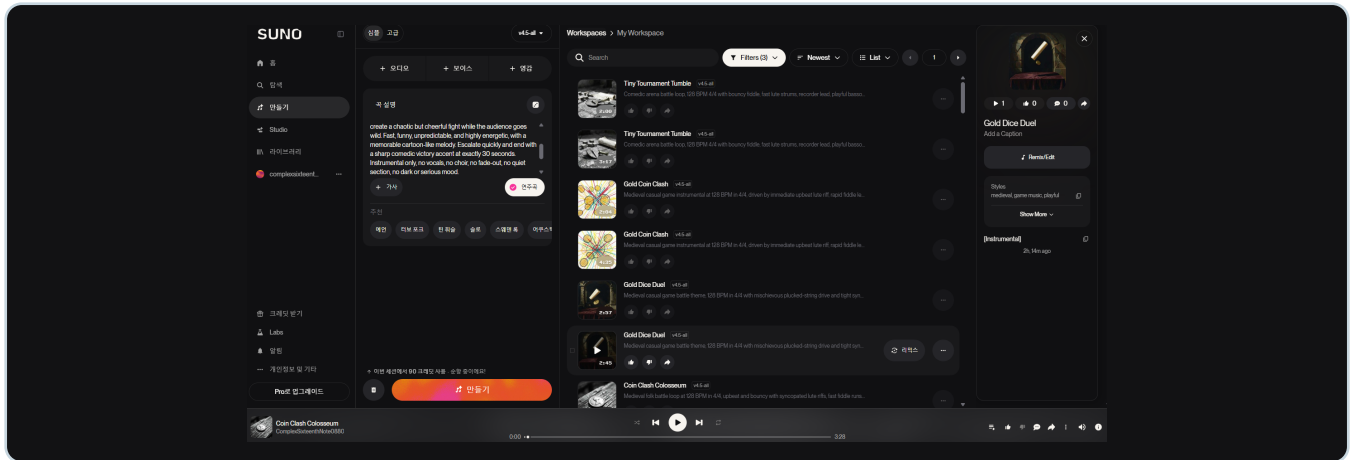
게임 장면과 길이 조건을 음악 생성 프롬프트로 구체화



대표 전투 BGM 프롬프트

Exactly 30-second instrumental battle loop for a colorful medieval casual mobile game. Start immediately with no intro. Energetic lute, fast fiddle, frame drums, hand claps and bright brass. Cute gladiators fight while spectators cheer and bet gold. Instrumental only, no vocals, no fade-out, no dark fantasy.

단순히 '중세 음악'을 요청하면 어둡고 웅장한 곡이 나오기 쉬워 밝은 모바일 캐주얼 게임, 축제형 콜로세움, 귀여운 검투사, 금화를 거는 관중이라는 장면 정보를 함께 제공하였다. 타이틀은 장난스러운 주제 선율, 배팅은 동전 소리와 계산하는 긴장감, 전투는 즉시 시작되는 빠른 리듬을 중심으로 구분하였다.



사람이 담당할 최종 판단

음악 생성 서비스가 요청 길이를 정확히 지키지 않을 수 있으므로, 실제 장면에 사용할 구간 선택과 30초 편집은 개발자가 수행하도록 계획하였다. 현재 저장소에서 음원 파일은 확인되지 않아 이 문서에서는 프롬프트 설계와 후보 생성 과정까지만 확정적으로 기술한다.

SOURCES & CONCLUSION

외부 에셋·오픈소스 출처와 결론

확인 가능한 출처는 명시하고, 확인되지 않은 정보는 추정하지 않음

구분	명칭	사용 목적	출처·비고
오픈소스	DOTween	UI·연출 Tween	dotween.demigiant.com DOTween 라이선스
상용 확장	DOTween Pro	Animation-Path 확장	dotween.demigiant.com/pro.php 유료 라이선스 필요
오픈소스	MCP for Unity	AI·Unity 연동	github.com/CoplayDev/unity-mcp MIT
오픈소스	SerializeReference Extensions	Inspector 지원	github.com/mackysoft/ Unity-SerializeReferenceExtensions
외부 에셋	Pixel Sprite Effects pack	전투 VFX	프로젝트 내 사용 원 구매 기록 기준
외부 에셋	RPG Icons Pixel Art	스킬·UI 아이콘	프로젝트 내 사용 원 구매 기록 기준
AI 생성	ChatGPT 아이콘	재화·메뉴·능력치	생성 기록 선별·후편집
AI 생성	SUNO BGM 후보	타이틀·배팅·전투	생성 기록 후보 비교·편집 필요

결론

AI를 사용하며 가장 크게 줄어든 것은 코드 입력 시간이 아니었다. 문제가 어느 파일에서 시작되는지 찾는 시간, 여러 Prefab을 같은 규칙으로 수정하는 시간, 테스트 상태를 반복해서 만드는 시간, 예외 상황과 회귀 조건을 정리하는 시간이 줄었다.

AI가 만든 결과는 코드·테스트·Unity 실행으로 확인했고, 검증된 결과만 통합하였다. AI는 개발자를 대신하기보다 판단 전에 필요한 조사와 반복 작업을 맡아 주는 개발 보조 도구로 가장 유용했다.

최종 흐름

문제·아이디어 정의 -> AI 조사·초안 -> 팀원 판단 -> 구현·생성 -> 테스트·실행 확인 -> 피드백 수정 -> 검증된 결과만 통합

제공 이미지 수록 확인

1 게임 대표 이미지	1쪽	2 Obsidian 목차	3쪽
3 Obsidian 내용	4쪽	4 SaveDebugTool	7쪽
5 인게임 전투	6쪽	6 드래그 범위	7쪽
7 인게임 로딩	2쪽	8 인게임 배팅	8쪽
9 인게임 상단 UI	8쪽	10 ChatGPT BGM	9쪽
11 SUNO 페이지	9쪽		